SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1402 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct:

I V.VAMSIDAHR REDDY (192365029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
V.VAMSI

Annexure A

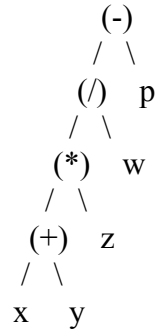
DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization
Given the three-address code:
t1 = x + y
t2 = t1 *
z t3 = t2 /
w t4 = t3
- p
Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Given three-address code:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 – p

Analysis
The operations are performed sequentially:
1. t1 = x + y
2. t2 = t1 * z
3. t3 = t2 / w
4. t4 = t3 - p

The equivalent mathematical expression is:
Equivalent Expression Tree



Role of Temporary Variables
Temporary variables such as t1, t2, t3, and t4 store intermediate results. They make the intermediate code easier for the compiler to generate, analyze, and optimize.
They also avoid repeatedly evaluating the same intermediate expression. For example, once x + y is calculated and stored in t1, its result can be reused instead of calculating it again.

Thus, temporary variables provide:
- Clear representation of individual operations.
- Easier register allocation.
- Better optimization opportunities.
- Reduced repeated computations.
- Simplified code generation.

Q2. Symbol Table and Memory Allocation
Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Given the following symbol table entries:

- p – char – 1 byte
- q – double – 8 bytes
- r – int – 4 bytes
- s – float – 4 bytes

A **symbol table** is a data structure maintained by the compiler to store information about variables, such as their names, data types, sizes, scope, and memory locations. During memory allocation, the compiler uses the information in the symbol table to determine how much memory each variable requires.

The variable p is of type char. A character generally requires **1 byte** of memory. The variable q is of type double, which requires **8 bytes** of memory. The variable r is an int, which generally requires **4 bytes** of memory. The variable s is of type float, which also generally requires **4 bytes** of memory.

Therefore, the total memory required without considering alignment or padding is:
Total memory = 1 + 8 + 4 + 4 = 17 bytes
Hence, the variables require a minimum of **17 bytes** of memory.

Memory Allocation
The compiler assigns memory locations to variables based on their data types and sizes. Each variable is stored at a particular memory address. The starting address of one variable depends on the size and alignment requirements of the previous variables.
For example, p requires only 1 byte, so it can be stored at any suitable byte address. However, q, being a double, commonly requires an address aligned to an 8-byte boundary. Similarly, r and s, being 4-byte types, commonly require 4-byte aligned addresses.

Effect of Alignment and Padding
Alignment means placing a variable at a memory address that is suitable for its data type. Proper alignment allows the processor to access data efficiently.
Sometimes, the compiler inserts additional unused bytes called **padding** between variables to satisfy alignment requirements. For example, if a double variable must begin at an address divisible by 8, some unused bytes may be inserted before it.

Because of padding, the actual memory occupied by these variables may be greater than the calculated 17 bytes. The exact amount depends on the compiler, processor architecture, and order in which the variables are stored.

The order of variables can also affect memory usage. Arranging variables according to their alignment requirements may reduce unnecessary padding. For example, larger-alignment

data types such as double are often placed before smaller data types.

Conclusion

The basic memory requirement for p, q, r, and s is **17 bytes** without padding. However, due to alignment requirements, the actual memory allocation can be greater than 17 bytes. Data types therefore influence not only the amount of memory allocated but also the organization and alignment of variables in memory. Proper memory alignment improves the efficiency of data access and contributes to better program performance.

Q3. Optimization in Intermediate Code

Given the intermediate code:

t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 -
t3

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Given the intermediate code:

t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3

Intermediate code is a representation of a program that is generated by the compiler before producing the final machine code. Optimization of intermediate code is performed to reduce unnecessary operations, improve execution speed, and use computer resources efficiently.

Step-by-Step Execution

The given code contains four instructions.

Step 1:

t1 = m * n

The values of m and n are multiplied, and the result is stored in the temporary variable t1.
Therefore:
t1 = m × n

Step 2:

t2 = p + q

The values of p and q are added, and the result is stored in t2.
Therefore:
t2 = p + q

Step 3:

t3 = m * n

The compiler again calculates m × n and stores the result in t3.
However, the expression m × n was already calculated in Step 1 and stored in t1. Therefore, this multiplication is a **redundant computation**.

Step 4:

t4 = t2 - t3

The value stored in t3 is subtracted from the value stored in t2.

Therefore:
t4 = (p + q) - (m × n)

Common Subexpression Elimination
The expression:
m * n
appears twice in the intermediate code:
t1 = m * n
t3 = m * n
This is called a **common subexpression**.

Since the value of m × n does not change between the two statements, the compiler does not need to calculate it twice. The previously calculated result in t1 can be reused.
The optimized intermediate code becomes:
t1 = m * n
t2 = p + q
t4 = t2 - t1
Here, the second multiplication has been completely eliminated.

Redundant Computation Removal
In the original code, m × n is performed twice. Performing the same calculation repeatedly wastes processor time.
After optimization, the multiplication is performed only once. The result stored in t1 is reused when calculating t4.
Thus, one unnecessary arithmetic operation is removed.

Benefits of Optimization
Intermediate-code optimization provides several advantages:

1. **Reduces execution time:** The multiplication m × n is calculated only once.
2. **Reduces the number of instructions:** The unnecessary instruction t3 = m * n is removed.
3. **Improves CPU efficiency:** The processor performs fewer arithmetic operations.
4. **Reduces temporary variables:** The temporary variable t3 is no longer required.
5. **Improves overall program performance:** Fewer instructions generally result in faster execution.
6. **Reduces resource usage:** The optimized code requires fewer computational resources.

Optimized Code
Original code:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
Optimized code:
t1 = m * n
t2 = p + q
t4 = t2 - t1
Conclusion
The given intermediate code contains a common subexpression, m × n, which is calculated twice. By applying **common subexpression elimination** and **redundant computation removal**, the compiler calculates the expression only once and reuses its result. This reduces the number of instructions and improves execution efficiency without changing the final output of the program.

Q4. Control Flow and Boolean Expression Handling
Given the Boolean expression:
if (x > y || p < q)
Intermediate
code:
if x > y goto
L1 if p < q
goto L1 goto
L2
L1: ...
Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Given the Boolean expression:
if (x > y || p < q)
The corresponding intermediate code is:
if x > y goto L1
if p < q goto L1
goto L2

Introduction
Boolean expressions are used in programming languages to make decisions and control the flow of execution. The logical OR operator || returns true if at least one of its two conditions is true.
In the given expression:
x > y || p < q
there are two conditions:

1. x > y
2. p < q

The complete expression is true if either condition is true. It is false only when both conditions are false.

Step-by-Step Control Flow
The first instruction is:
if x > y goto L1
The compiler first evaluates the condition x > y.

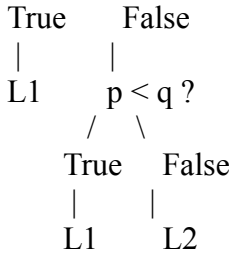
- If x > y is **true**, control immediately jumps to label L1.
- If x > y is **false**, execution continues to the next instruction.

The second instruction is:
if p < q goto L1
If the first condition was false, the compiler evaluates p < q.

- If p < q is **true**, control jumps to L1.
- If p < q is **false**, execution continues to the next instruction.

The third instruction is:
goto L2
If both conditions are false, control moves to L2.
Therefore, the control flow can be represented as:

```
        x > y ?
       /      \
```



Short-Circuit Evaluation

The logical OR operator uses **short-circuit evaluation**.
Short-circuit evaluation means that the second condition is evaluated only when it is necessary.
For the expression:
x > y || p < q
if x > y is true, the entire expression is already known to be true. Therefore, there is no need to check p < q.
For example, if:
x = 10
y = 5
then x > y is true. The compiler directly jumps to L1 without evaluating p < q.
If x > y is false, then the compiler must evaluate p < q to determine the final result.

Cases of Execution

Case 1: Both conditions are true
If x > y is true, control immediately goes to L1. The second condition is not checked.

Case 2: First condition is false and second condition is true
If x > y is false, the compiler evaluates p < q. Since it is true, control goes to L1.

Case 3: Both conditions are false
If x > y is false and p < q is also false, control reaches goto L2.

Advantages of Short-Circuit Evaluation

Short-circuit evaluation provides several benefits:

1. **Reduces unnecessary condition checking.**
2. **Improves execution speed** by avoiding unnecessary comparisons.
3. **Reduces the number of instructions executed.**
4. **Provides efficient control flow** in Boolean expressions.
5. **Helps the compiler generate optimized intermediate code.**

Conclusion
The given intermediate code efficiently implements the Boolean expression x > y || p < q. The compiler first checks x > y. If it is true, execution immediately jumps to L1. Otherwise, it checks p < q. If both conditions are false, control goes to L2. This is an example of **short-circuit evaluation**, which avoids unnecessary condition checking and improves execution efficiency.

Q5. Backpatching and Flow Control
Given:

Instruction	Target

200: if x==y goto _
201: goto _
Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Given the following intermediate code:
200: if x == y goto _
201: goto _
The backpatch lists are:
truelist = {200}
falselist = {201}

Introduction
Backpatching is an important technique used by a compiler during intermediate code generation. It is mainly used for handling Boolean expressions and control-flow statements such as if, if-else, while, and for loops.
During code generation, the compiler may not know the exact target address of a jump instruction. Therefore, it temporarily leaves the target address blank. Later, when the target address becomes known, the compiler fills in the missing address. This process is called **backpatching**.

Given Intermediate Code
The given code is:
200: if x == y goto _
201: goto _
Here, the underscore _ represents an unknown target address.
Instruction 200 checks whether x == y.

- If the condition is **true**, control should go to the true branch.
- If the condition is **false**, control should go to the false branch.

The compiler does not yet know where these branches will begin, so the target addresses are left blank.

True List
The given true list is:
truelist = {200}
This means instruction 200 must eventually be connected to the target address that represents the **true condition**.
For example, suppose the true branch begins at instruction **250**.
After backpatching:
200: if x == y goto 250
Therefore, the target address of instruction 200 is updated to 250.

False List
The given false list is:
falselist = {201}
This means instruction 201 must be connected to the target address representing the **false condition**.
Suppose the false branch begins at instruction **300**.

After backpatching:
201: goto 300
Therefore, the target address of instruction 201 is updated to 300.

Final Intermediate Code
After applying backpatching, the code becomes:
200: if x == y goto 250
201: goto 300
Here:

- 250 is the destination when x == y is true.
- 300 is the destination when x == y is false.

Working of Backpatching
The process can be explained in the following steps:
Step 1: The compiler generates conditional and unconditional jump instructions.
200: if x == y goto _
201: goto _

Step 2: The compiler places instruction 200 in the truelist because it represents the true outcome.
truelist = {200}

Step 3: Instruction 201 is placed in the falselist because it represents the false outcome.
falselist = {201}

Step 4: When the compiler determines the actual target addresses, it replaces the blank targets.
For example:
truelist → 250
falselist → 300

Step 5: The final code becomes:
200: if x == y goto 250
201: goto 300

Importance of Backpatching
Backpatching is important because the compiler does not always know the destination of a jump when the intermediate code is initially generated.
It is useful for:

1. Translating Boolean expressions.
2. Handling conditional statements.
3. Implementing if-else statements.
4. Implementing loops such as while and for.
5. Managing conditional and unconditional jumps.
6. Connecting different parts of intermediate code.
7. Supporting efficient control-flow generation.

Backpatching in Boolean Expression Translation
Backpatching is especially useful when Boolean expressions contain operators such as AND, OR, and NOT. The compiler maintains lists of instructions whose target addresses are not yet known.

The two important lists are:

- **Truelist:** Contains instructions that should jump when the Boolean expression is true.
- **Falselist:** Contains instructions that should jump when the Boolean expression is false.

Once the target locations are determined, the compiler fills in the missing addresses using backpatching.

Conclusion

Backpatching is a technique used to complete incomplete jump instructions during intermediate code generation. In the given example, instruction 200 belongs to the truelist, while instruction 201 belongs to the falselist. If the true and false target addresses are 250 and 300 respectively, backpatching changes the code to:

200: if x == y goto 250

201: goto 300

Thus, backpatching plays an important role in Boolean expression translation and control-flow handling by allowing the compiler to determine jump targets at the appropriate stage of code generation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand